

第一讲 作业:

1.

冒泡算法伪代码:

$n = \text{nums.length}$

for $i = 0 \rightarrow n-1$ // 执行 $n-1$ 次冒泡

for $j = 0 \rightarrow n-i-1$ // 每次冒泡比较 $n-i$ 次

if $\text{nums}[j] > \text{nums}[j+1]$

swap($\text{nums}[j]$, $\text{nums}[j+1]$)

循环不变量: x 次冒泡后, 已通过冒泡排序好的部分,
包含 nums 中最大的 x 个数, 且已按照升序排列好。

证明正确性:

每次冒泡过程, 会将未排序中最大值移至最高处 (即下标最大处),

已排序部分递增排列。分三个阶段证明:

a. 初始化: 未开始冒泡, 0 个最大元素已排序, 故循环不变量初始为真

b. 保持: 设第 i 次冒泡迭代前, 循环不变量为真, 故包含 $i-1$ 个最大元素已排序。

第 i 次冒泡, 依次比较剩下的元素, 交换使得最大元素移至下标最大处。

得到 i 个最大元素已排序, 所以循环不变量保持为真。

c. 终止: 当迭代 $n-1$ 次后终止, 此时剩下最后一个元素 $\text{nums}[0]$ 处于下标最小处, 该元素自然为最小元素

因此, 整个数组已完全排序, 循环不变量为真。

2.

要证明: 如果 $n < F_{k+1}$, 那么 Euclid(m, n) 执行次数 $< k$.

转换为逆否命题: 如果 Euclid(m, n) 执行次数为 k , 则 $n \geq F_{k+1}$.

令 $r_0 = m, r_1 = n$, 则可得序列: (假设 $m \geq n$).

$$r_0 = q_1 r_1 + r_2$$
$$r_1 = q_2 r_2 + r_3$$
$$\dots$$
$$r_{k-1} = q_k r_k + 0 \quad (r_i, q_i \text{ 均为正整数}).$$

针对每段序列, 可得出新命题: $r_j \geq F_{j+2} \quad (0 \leq j < k)$. $\because$ 当取最大值 $k-1$ 时, $r_{k-1} \geq F_{k+2}$, 即 $n \geq F_{k+2}$. 故可证.

① $j=0, F_2=1$.
 $\because r_1 = n \geq 1 \therefore r_0 \geq 1$
故 $r_0 \geq F_2$

② $j=1, F_3=2$.
 $\because$ 执行至 $r_{k-1} = q_k r_k$, 故 $r_{k-1} \neq r_k$.
 $\therefore q_k \geq 2$.
 $\therefore r_{k-1} = q_k r_k + 0$. 且 $r_{k-2} \geq 1$
 $\therefore r_{k-1} \geq 2$
故 $r_{k-1} \geq F_3$.

假设: V 非空集合 i ; 记 $j, j \geq 2$, 命题为真.

要证: $r_{k-j} \geq F_{j+2}$.

$$\text{由 } r_{k-j} = r_{k-j+1} \cdot r_{k-j+1} + r_{k-j+2}$$

$$\geq r_{k-j+1} + r_{k-j+2}$$

$$\geq F_{j+1} + F_j$$

$$\geq F_{j+2}$$

$\therefore$ 得证

3.

Donald Knuth 的事迹:

a. 著作: 《计算机程序设计艺术》(The Art of Computer Programming, TAOCP): 计算机科学的"圣经"

从 60 年代末开始撰写, 计划写七卷, 已完成前四卷.

该书以算法研究为主线, 确立了算法作为计算机科学基础的重要主题。

b. Knuth 对算法的"渐近分析"的推广和系统化应用做出了开创性贡献。

他严格分析了大量算法的性能, 为算法效率评价提供了数学基础

c. 荣誉: Knuth 因其在算法分析和 TAOCP 上的卓越贡献, 于 1974 年获得图灵奖, 年仅 36 岁。

d. 在撰写 TAOCP 期间, Knuth 对当时书籍的排版质量非常不满。

为此, 他花了近十年时间, 发明了 TeX 排版系统和 METAFONT 字体设计系统。

TeX 因其对数学公式和复杂排版的出色处理能力, 至今仍是全球学术界撰写论文和书籍的标准工具。

个人思考: Knuth 的事迹给我的最大启发是"工匠精神"和"解决根本问题"的思维。

他不仅是计算机科学家, 还是一位数学家和排版艺术家。

为了写好一本书, 可以花十年时间发明一套排版工具, 这种对质量的极致追求令人敬佩。

第二讲 作业

1.

主要矛盾: 由 $O \setminus \Omega$ 和 Θ 刻画, 忽略低阶项和常数。

它决定了算法在大规模输入下的可行性. 是描述函数在极限附近行为的方法

次要矛盾: 常数项与低阶项是被渐近符号隐藏的细节。

它决定了算法在小规模输入或同阶算法竞争时的优劣。

辩证统一的应用:

宏观定性: 在大数据场景 (如 $n=10^9$), 主要矛盾占主导,

必须选 $O(n \log n)$ 而非 $O(n^2)$ 。

微观定量: 在小规模场景 (如 $n=50$) 或同阶比较 (如快排 vs 堆排),

次要矛盾转化为主要矛盾。例如, 虽然都是 $O(n \log n)$,

但快排因常数因子小, 实战通常更快。

2. 习题一

1.1.

Find(A): // A为元素极 > 2 的数组, 各元素值互异.

if $A[1] < A[2]$

if $A[2] < A[3]$ return $A[2]$

else if $A[1] < A[3]$ return $A[3]$ // $A[3] > A[2]$

else return $A[1]$

else // $A[2] > A[1]$

if $A[3] > A[2]$ return $A[2]$

else if $A[2] > A[1]$ return $A[1]$

else return $A[1]$

∴ 最多比较 3 次, 最少 2 次.

故最坏情况下比较 3 次.

1.4. 对层 while 循环是递归, $\log_2 n$ 次, 从 $n=2^t \rightarrow 0$.

故内层 for 循环是递归: $2^t + 2^{t-1} + \dots + 2^1 + 2^0 = \frac{2^t(1-2^{t+1})}{1-2} = 2^{t+1} - 1$ (次)

已知 $n=2^t$ ∴ 递归加法总次数: $2n-1$

1.6. $y = P[0] + P[1]x + P[2]x^2 + \dots + P[n]x^n$.

∴ 是求单多项式求值问题.

乘法 $2n$ 次, 加法 n 次.

1.13. i): 要证 $f(n) = O(g(n))$, $\forall \epsilon > 0 \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$

$$\lim_{n \rightarrow \infty} \frac{n^d}{n^n} \xrightarrow{\text{洛}} \lim_{n \rightarrow \infty} \frac{d \cdot n^{d-1}}{n^n \cdot n} = 0$$

继续使用洛必达法则, 处理 k 次. $k = \lfloor d+1 \rfloor$

$$\xrightarrow{\text{洛}} \lim_{n \rightarrow \infty} \frac{d(d-1)(d-2)\dots(d-k+1)n^{d-k}}{n^n (n^n)^k} = 0$$

∴ $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$. 故得 i):

1.17.

✓ ① $(n-2)! = \Theta((n-2)!)$

✓ ② $5 \log(n+100)^{10} = \Theta(\log n)$

✓ ③ $2^{2n} = \Theta(4^n)$

✓ ④ $0.001n^4 + 3n^3 + 1 = \Theta(n^4)$

✓ ⑤ $(\ln n)^2 = \Theta(\log^2 n)$

✓ ⑥ $3\sqrt{n} + \log n = \Theta(n^{\frac{1}{2}})$

✓ ⑦ $3^n = \Theta(3^n)$

✓ ⑧ $\log(n!) = \Theta(n \log n)$

✓ ⑨ $\log(n^{n^n}) = \Theta(n \log n)$

✓ ⑩ $1 + \frac{1}{2} + \dots + \frac{1}{n} = \Theta(\log n)$

排列: $5 \log(n+100)^{10} \approx 1 + \frac{1}{2} + \dots + \frac{1}{n}$

$< (\ln n)^2$

$< 3\sqrt{n} + \log n$

$< \log(n!) \approx \log(n^{n^n})$

$< 0.001n^4 + 3n^3 + 1$

$< 3^n$

$< 2^{2n}$

$< (n-2)!$

第三讲. 作业.

1. (1). 迭代法. $T(n) = T(1) + \sum_{i=2}^n i^2 = \frac{n(n+1)(2n+1)}{6}$

$$\Rightarrow T(n) = \Theta(n^3)$$

(2). 主定理. $n^{\log_3 9} = n^2$. $\therefore f(n) = n = O(n^{2-\varepsilon})$

$$\Rightarrow T(n) = \Theta(n^2).$$

(3). 递归树法.

$$\begin{array}{ccc} & cn & \rightarrow cn \\ & \swarrow \quad \searrow & \\ c\frac{n}{2} & & c\frac{n}{4} \quad \rightarrow c\frac{3}{4}n \\ \swarrow \quad \searrow & & \swarrow \quad \searrow \\ c\frac{n}{4} & c\frac{n}{8} & c\frac{n}{8} & c\frac{n}{16} \quad \rightarrow c(\frac{3}{4})^2 n \end{array}$$

$$\begin{aligned} \Rightarrow T(n) &= \sum_{i=0}^{\log_2 n} (\frac{3}{4})^i \cdot cn = \frac{1 \times (1 - (\frac{3}{4})^{\log_2 n})}{1 - \frac{3}{4}} \cdot cn = cn - cn^{(\log_2 3) - 1} \\ &= \Theta(n) \end{aligned}$$

△补充思路: 级数收敛总代价. 公比为 $\frac{3}{4} < 1$. $\therefore$ 该树根比叶重. 故 $T(n)$ 由根节点决定. 为 $\Theta(n)$.

(4). 迭代法: $T(n) = \sum_{i=1}^n \log^3 i$

$$\Rightarrow T(n) = \Theta(n \log^3 n)$$

(5) 主定理. $n^{\log_2 5}$. $f(n) = (n \log n)^2 = O(n^{(\log_2 5) - \varepsilon})$

$$\Rightarrow T(n) = \Theta(n^{\log_2 5}).$$

(6) 主定理. $n^{\log_2 2} = n$. $f(n) = n^2 \log n = \Omega(n^{1+\varepsilon})$

正则条件: $2 \left(\frac{n}{2}\right)^2 \log \left(\frac{n}{2}\right) \leq cn^2 \log n \Rightarrow c \geq \frac{1}{2} \left(1 - \frac{\log 2}{\log n}\right)$. 对于 $n \rightarrow \infty$.

$$\Rightarrow T(n) = \Theta(n^2 \log n).$$

取 $c \geq \frac{1}{2}$ 即可成立.

(7). 调和级数: $T(n) = \sum_{i=1}^n \frac{1}{i} = \Theta(\log n)$

(8). $T(n) = \sum_{i=1}^n \log i = \log(1 \times 2 \times \dots \times n) = \log(n!)$ 根据 Stirling 公式:

$$\Rightarrow T(n) = \Theta(n \log n).$$

2. 根据主定理:

算法A: $n^{\log_2 7}$. $f(n) = n^2 = O(n^{(\log_2 7) - \varepsilon})$

$$\Rightarrow T(n) = \Theta(n^{\log_2 7})$$

算法B: $n^{\log_4 a}$. $f(n) = n^2$. 情况一: $\log_4 a > 2$. 即 $a > 16$. $\Rightarrow W(n) = \Theta(n^{\log_4 a})$

要满足 $\log_4 a < \log_2 7$. 即 $a < 49$.

情况二: $a = 16$. $W(n) = \Theta(n^2 \log n) < \Theta(n^{\log_2 7})$. $\therefore$ 满足.

情况三: $a < 16$. $W(n) = \Theta(n^2) < \Theta(n^{\log_2 7})$. $\therefore$ 满足.

$\therefore$ 综上, $a_{\max} = 48$.

No.

DATE / /

3. 猜想: $T(n) \sim 2T(n/2) + 1 \Rightarrow T(n) = 2n - 1 = \Theta(n)$.要证: $T(n) = \Theta(n)$ 证明: 奠基: $n=1$, $T(1)=1$, 满足.归纳: $T(n) = O(n)$ $T(n) = \Omega(n)$ 归纳假设: $\forall k < n$, 有 $T(k) = 2k - 1$. $T(n) \leq cn$ $T(n) \geq cn$ 归纳步骤: $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$ $T(n) \leq c\lfloor \frac{n}{2} \rfloor + c\lceil \frac{n}{2} \rceil + 1$

$$= (\lfloor n/2 \rfloor - 1) + (\lceil n/2 \rceil - 1) + 1 = cn + 1 \quad \text{X 无法继续}$$

$$= 2(\lfloor n/2 \rfloor + \lceil n/2 \rceil) - 1$$

$$T(n) \geq cn + 1 > cn \quad \checkmark$$

$$= 2n - 1$$

改用更强的归纳假设:

 $\therefore$ 得证.

$$T(n) \leq cn - b$$

$$T(n) \leq c\lfloor \frac{n}{2} \rfloor - b + c\lceil \frac{n}{2} \rceil - b + 1$$

4. 偶数 $2n$ 个: $J(2n) = 2J(n) - 1$. // 第1轮去除 $2, 4, \dots, 2n$, 留下 n 个 $1, 3, \dots, 2n-1$. $= cn - 2b + 1$ 奇数 $2n+1$ 个: $J(2n+1) = 2J(n) + 1$.新编号: $1, 2, \dots, n \leq cn - b$

$$(b \geq 1)$$

规律: $J(1) = 1$.

$$J(2) = 1, J(3) = 3.$$

$$J(4) = 1, J(5) = 3, J(6) = 5, J(7) = 7.$$

$$J(8) = 1.$$

 $\Rightarrow n$ 为 2 的幂时, $J(n) = 1$, 其后依次 $+2$.记 $n = 2^m + l$, ($0 \leq l < 2^m$).

$$J(n) = 2l + 1.$$

证明: 奠基: $n=1 = 2^0 + 0$, $J(1) = 2 \cdot 0 + 1 = 1$, 满足.归纳假设: $\forall k < n$, 有 $T(k) = 2l + 1$.归纳步骤: n 为偶, $n = 2k$, $k = 2^m + l$.

$$J(n) = 2J(k) - 1.$$

$$= 2(2l + 1) - 1$$

$$= 4l + 1$$

$$\because n = 2k = 2^{m+1} + 2l \quad \therefore J(n) = 4l + 1 \quad \therefore \text{一致.}$$

 n 为奇, $n = 2k + 1$, $k = 2^m + l$

$$J(n) = 2J(k) + 1$$

$$= 2(2l + 1) + 1$$

$$= 4l + 3$$

$$\because n = 2k + 1 = 2^{m+1} + 2l + 1 \quad \therefore J(n) = 4l + 3 \quad \therefore \text{一致.}$$

 $\therefore$ 综上, 得证.

第四讲. 作业.

1. 双指针法. 左指针指向原数序列末尾, 右指针向右遍历.

右指针遇到负值, 移至左指针处.

$\Rightarrow$ 时间复杂度: $O(n)$. 仅遍历一遍.

空间复杂度: $O(1)$. 仅使用常数个额外变量.

(1) 伪代码.

2. function DoubleHanoi (n, A, B, C):

if $n == 1$:

$A \rightarrow C$

$A \rightarrow C$

else:

DoubleHanoi ($n-1, A, C, B$)

$A \rightarrow C$

$A \rightarrow C$

DoubleHanoi ($n-1, B, A, C$)

(2). $T(n) = 2T(n-1) + 2$. $T(1) = 2$.

$$= 2^{n-1} T(1) + \sum_{i=1}^{n-1} 2^i$$

$$= 2^{n+1} - 2.$$

3. 二分法. 对数组进行递归二分.

每步二分. 判断中间值与两侧值. 若 $left > mid$, 对左半数组继续二分.

分类讨论

else if $left < mid < right$, return mid.

else. 对右半数组继续二分.

$\Rightarrow$ 时间复杂度: $O(\log n)$.

4. 双指针法. 设置指针 $i=1, j=n$;

while ($i < j$):

sum = $A[i] + A[j]$;

if (sum == x) return true;

else if (sum < x) $i++$;

else $j--$;

return false;

$\Rightarrow$ 时间复杂度: $O(n)$

空间复杂度: $O(1)$.

$$\begin{array}{l} A[i] + A[j] \\ \swarrow \quad \searrow \\ \text{min} \quad \text{max} \end{array} \left\{ \begin{array}{l} = x. \checkmark \\ > x. A[j] \times \\ < x. A[i] \times \end{array} \right.$$

No.

DATE / /

第五讲. 作业.

(1). 2.6. ① 遍历每一行, 使用二分法查找最后一个1的位置, 记录最多1的行.

n 行, 每行 $\log n \Rightarrow T(n) = O(n \log n)$.

② r 为行指针, c 为列指针. r 从1移至 n 遍历 n 行.

c 从1向后移动, 始终指向已访问行中存在1的最大列. (c 只增不减).

2.7. ① 首先进行排序, 最优复杂度 $O(n \log n)$. 遍历数组, 对元素计数即可得到主元素. $O(n)$ 复杂度.

$\Rightarrow T(n) = O(n \log n)$.

优化: 如果存在主元素, 一定为 $A[n/2]$.

② 直接遍历数组, 记录每个元素值的个数. (空间换时间). 当一个元素数达到 $\lceil n/2 \rceil$ 为主元素.

$\Rightarrow T(n) = O(n)$.

③ 分治法: 二分数组, 递归查找两侧主元素 L 和 R .

if. $L == R$. return L

elif. 只有 L , 在整个数组中统计 L . 过半则返回, 否则返回无

diff. 只有 R . 同上.

else. 返回无.

$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$.

2.8. 分治法: 比较2数组的中位数 m_A, m_B

if $m_A == m_B$ return m_A

elif $m_A > m_B$ 总体中位数位于 A 左半边和 B 右半边. 则可删去一半.

else $m_A < m_B$... A 右半边和 B 左半边. ...

$\therefore$ 每次规模减半.

$T(n) = O(\log n)$.

(2). 补0. 通过对矩阵填充0, 使其维度扩展至最近的 2^k .

算法复杂度不变, 但时空复杂度都会增加.

(3). 在递归开始前, 就对所有数组按 γ 值升序排列得到 P_r .

在分治阶段, 先根据 P_r 找到中位数进行切分, 再遍历 P_r . 根据其 γ 值分配给

L_r 和 R_r 在递归过程中表现为子问题的 P_r .

在合并阶段, P_r 已有序可直接合并.

$\therefore T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$.

第六讲. 作业.

(1). memo = array of size $n+1$, init with -1.

function Fibonacci(n):

if $n \leq 1$: return n ;

if memo[n] $\neq -1$, return memo[n];

memo[n] = Fibonacci[$n-1$] + Fibonacci[$n-2$];

return memo[n];

(2). 递推方程: $r_n = \max_{1 \leq i \leq n} (P_i + r_{n-i})$. $\rightarrow$ if $i=n$, $r_n = P_n$
 else $r_n = \max (P_i + r_{n-i} - c)$

$$\Rightarrow r_n = \max (P_n, \max_{1 \leq i < n} (P_i + r_{n-i} - c))$$

(3). 仍存在最优子结构, 因为问题结构未改变。

修改方法: 仅需将求 min 改为求 max. γ :

$$m[i, j] = \max_{i \leq k \leq j} (m[i, k] + m[k+1, j] + P_i P_k P_j).$$

DATE _____

DATE _____

第七讲. 作业

1. ③.8.

若 N 为奇, 则不可能有解; 故考虑 N 为偶:

1. 刻画最优子结构: 寻找子集元素和 $W = N/2$. 假设 A 前 i 个元素中存在一个和为 j 的子集. 要么前 $i-1$ 个中已包含该子集, 要么加上第 i 个才构成该子集.

2. 递归定义最优值: $\text{bool } dp[i][j]$: 前 i 个数中存在和为 j 的子集.

初始: $dp[0][0] = \text{True}$; $dp[0][j] = \text{False}$ ($j > 0$)

递归方程: $dp[i][j] = dp[i-1][j] \vee dp[i-1][j-a_i]$, when $j \geq a_i$
 $dp[i-1][j]$, when $j < a_i$

3. 计算最优值:

外层 i 从 $1 \rightarrow n$, 内层 j 从 $1 \rightarrow W = N/2$, 内部根据 j 与 a_i 大小关系选择处理.

时间复杂度: $O(n \cdot N)$

4. 构造最优解:

if $dp[n][W] = \text{True}$, 则说明有解, 回溯表格打印方案.

② 3.9. (类似上题 3.8, 同为 0-1 背包问题)

1. 刻画最优子结构: 从前 i 个物品中选择总时间 $\leq j$ 向最大的总重量.

2. 递归定义最优值: 初始 $V[0][j] = 0$

递归: $V[i][j] = \begin{cases} \max\{V[i-1][j], V[i-1][j-t_i] + v_i\}, & \text{when } j \geq t_i \\ V[i-1][j] & \text{when } j < t_i \end{cases}$

3. 计算最优值: for $i = 1 \rightarrow n$:

for $j = 0 \rightarrow D$:

if $t[i] \leq j$:

$V[i][j] = \max(V[i-1][j], V[i-1][j-t[i]] + v[i])$

else:

$V[i][j] = V[i-1][j]$

时间复杂度: $O(n \cdot D)$.

4. 构造最优解: 从 $V[n][D]$ 回溯, if $V[i][j] > V[i-1][j]$, 则将前 i 个物品选中.
 $j = j - t_i$.

③ 3.12.



1. 构造最优子结构: $A[i, i+1, \dots, j]$ 的最优剖分, 必然包含以边 $V_i V_j$ 为底 $\Delta V_i V_k V_j$

则 A 被分为两部分: $A[i, i+1, \dots, k]$ 和 $A[k, k+1, \dots, j]$.

A 的最优值: A , 而最优值 $= A_1$ 的最优值 $+ A_2$ 的最优值 $+ C \Delta V_i V_k V_j$.

2. 递归求最优值: $t[i][j]$ 记录 $i \rightarrow j$ 的边所形成的最优剖分取值.

初始: 当 $j = i+1$, $t[i][j] = 0$.

递推: $t[i][j] = \min_{i < k < j} \{ t[i][k] + t[k][j] + C \Delta V_i V_k V_j \}$

其中 $C \Delta V_i V_k V_j = d_{ik} + d_{kj} + d_{ij}$.

3. 计算最优值.

for $r = 2 \rightarrow n+1$: // 链长度, 从小到大, 从只考虑前3个点开始.

for $i = 1 \rightarrow n+1$:

$j = i+r$

$t[i][j] = +\infty$

for $k = i+1 \rightarrow j-1$:

$q = t[i][k] + t[k][j] + C(i, k, j)$

if $q < t[i][j]$:

$t[i][j] = q$

$sc[i][j] = k$ // 记录最优剖分节点.

时间复杂度: $O(n^3)$.

4. 构造最优解: 利用上述的 $sc[i][j]$ 记录构造.

2. 利用公式: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$. 与杨辉三角递推公式一致. 令 $dp[i][j]$ 表 $\binom{i}{j}$.

初始: $dp[i][0] = 1$, $dp[i][i] = 1$.

Function Combination(n, k):

if $k > n/2$: $k = n - k$ // 利用对称性.

$dp = \text{array}[0..k]$ 初始为 0.

$dp[0] = 1$

for $i = 1 \rightarrow n$:

for $j = \min(i, k) \rightarrow 1$:

$dp[j] = dp[j] + dp[j-1]$

return $dp[k]$

时间复杂度: $O(n^2)$.

3. $L[i]$ 表示以 a_i 结尾的最长递增子序列长度

$$L[i] = 1 + \max(\{L[j] \mid 1 \leq j < i, a_j < a_i\}, 0)$$

求得 $\max_{1 \leq i \leq n} (L[i])$.

时间复杂度: $O(n^2)$.

4. $F(i, y) = \max\{F(i-1, y), F(i-1, y-w_i) + v_i\}$.

可见, 计算 i 行时, 只需要 $(i-1)$ 行的数据, 故可将二维表转化为一维. $dp[0..b]$

Function bag (W, V, b, n):

$dp = \text{array}[0..b]$ 初始为 0

for $i = 1 \rightarrow n$:

$w = W[i]$

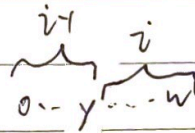
$v = V[i]$

for $y = b \rightarrow w$:

$dp[y] = \max(dp[y], dp[y-w] + v)$

return $dp[b]$.

空间复杂度: $O(b)$.



第八讲. 作业.

1. Function `bag(weights, values, W, n):`for $i=1 \rightarrow n$: $r[i] = \text{values}[i] / \text{weights}[i]$ Sort items by r descending;total- $v=0$ current- $w=0$ for $i=1 \rightarrow n$:if $\text{current-}w + \text{weights}[i] \leq W$:total- $v += \text{values}[i]$ current- $w += \text{weights}[i]$

else:

remain = $W - \text{current-}w$ total- $v += \text{values}[i] * (\text{remain} / \text{weights}[i])$

break;

return total- v

正确性证明: 假设最优解未包含最高性价比物品 i 的最大可能量, 则可以从其中拿出一些其他物品来替换成 i . 因 $v_i/w_i > v_j/w_j$, 这一替换始终有益. 故与最优解矛盾.
 因此先装最高性价比的物品, 保证了算法的正确性.

2. 习题 4.4. Function `MinIntervals(p):`sort(p) ASC

count = 0

 $i=0$ while $i < n$: // 第 i 个区间覆盖最左侧的点, 让左端点落在该点上, 右端点可落得尽可能远。

count++

end- $p = p[i] + 1$ while $i < n$ & $p[i] \leq \text{end-}p$

return count++

时间复杂度: $O(n \log n)$.

正确性证明: 对于集合中最左侧的点, 必被区间覆盖.

贪心选择左端点为 a_i , 区间为 $[a_i, a_i+1]$.

使得区间长度尽可能短, 覆盖尽可能多的点.

假设 $[a_i, a_i+1]$ 非最优解的第一个区间, 右端点要大于 a_i , 左端点可能让最左侧点移出区间范围.

故该贪心算法安全.

No.

DATE / /

3. Function Is Subsequence (X, Y):

$i=0, j=0$

while $i < \text{len}(X)$ & $j < \text{len}(Y)$:

if $X[i] == Y[j]$:

$i=i+1$ // 匹配成功.

$j=j+1$ // 匹配失败.

return $i == \text{len}(X)$

正确性证明: 假设 Y 中第一个可以匹配 $X[0]$ 的位置为 j_0 .

若选择 $j > j_0$ 作为 $X[0]$ 的匹配点.

Y 在 j_0 后的字符串长度 $<$ 在 j 后的, 则可以匹配上 X 的可能性降低.

故贪心选择更安全.

第9讲作业.

4.7. (1) Function Greedy Change (Coins, y);

已有 Coins = $[p^n, p^{n-1}, \dots, p, 1]$

count[0..n] 初始为 0

for $i \leftarrow 0$ to n do $C = \text{Coins}[i]$ count[i] = $\lfloor y/C \rfloor$ // 从大面额开始取, 向下取整. $y = y \% C$

return count

(2). 正确性证明. (反证法).

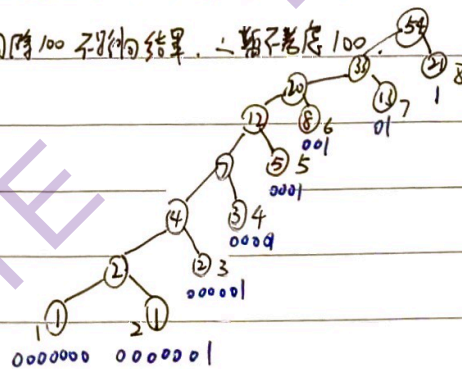
i1: 假设贪心解 A 非最优, 则存在最优解 A'.

 $\therefore$ during 贪心, 对 p^k 最多取 $(p-1)$ 个. $\therefore$ 不存在用大面额替换多个小面额的可能.则必然存在某 p^k 小于 A 中的, 而这部分须由小面额补齐A' 中 $\sum_{i=0}^{k-1} (p-1)p^i = p^k - 1 < p^k$, $\therefore p^k$ 无法由小面额补齐

故矛盾, 无法通过减少大面额增加小面额来实现

 $\therefore$ 贪心为最优解.

4.12. (1). Fibonacci 前 8 项: 1, 1, 2, 3, 5, 8, 13, 21

 $\therefore$ 构造时同乘同时 100 不影响结果. $\therefore$ 暂不考虑 100.(2). 深度为 $(n-1)$ 的单调递减树.i1: 第 k 步后, 剩余节点为 $\{S_k, F_{k+2}, F_{k+3}, \dots\}$.已知 $F_{k+2} < F_{k+3}$, 若 $S_k < F_{k+3}$, 则最小节点为 S_k 与 F_{k+2} . $\therefore$ 下一步一定会合并这两个, 形成单链.

No.

DATE / /

基础步骤: $k=1$. $S_1 = F_1 + F_2 = 2 < F_{1+3} = F_4 = 3$. 成立

$k=2$. $S_2 = S_1 + F_3 = 2 + 2 = 4 < F_{2+3} = F_5 = 5$. 成立

归纳假设: $(k-1)$ 步时成立, $\forall S_{k-1} < F_{k+2}$.

归纳递推: $S_k = S_{k-1} + F_{k+1}$

要证 $S_k < F_{k+3}$. $\forall k$. $S_{k-1} + F_{k+1} < F_{k+3}$

$$\therefore F_{k+3} = F_{k+2} + F_{k+1}$$

$\therefore \forall k$. $S_{k-1} < F_{k+2}$. 为假设, 为真.

$\therefore$ 得证.

4.15. A 仅 1 台, n 项任务只能串行执行, 总用时 $T_A = \sum_{i=1}^n a_i$.

而 B 有任意台, $\therefore T_B$ 取决于 B 上加工顺序.

故, 可以将任务按 b_i 长任务优先进行执行.

正确性证明:

假设不按 b_i 降序排列, 则存在相邻任务 i, j , 满足 $b_i < b_j$, 则:

$$\text{交换前: } T_1 = T' + a_i + a_j + b_j$$

$$\text{后: } T_2 = \max \left\{ \underbrace{T' + a_j + a_i + b_i}_{\text{case 1}}, \underbrace{T' + a_j + b_j}_{\text{case 2}} \right\}$$

$$\text{case 1. } \because b_i < b_j \therefore T_2 < T_1$$

$$\text{case 2. } \therefore T_2 = T_1 + b_j - b_i < T_1$$

综上, $T_2 < T_1$. 与假设矛盾.

故得证.

最坏情况下: $O(n \log n)$

4.17. (1). 该树为最大生成树 MST.

证: 设 MST 为 T , $\forall u, v$ 间存在唯一路径 P_1 .

假设原图中有另条 P_2 . s.t. $w(P_2) > w(P_1)$

将 P_2 的边加入 T 后, 会形成回路.

根据 MST 性质, MST 外其他边一定是回路中最小的.

故矛盾.

$\therefore$ 得证.

(2). 算法: 修改的 Kruskal 算法:

① 将图中边按权值从大到小排序.

② 初始化并查集. 每个节点为独立.

③ 遍历排序后的边 (u, v) :

若 u, v 不连通, 则加入到 T , 并合并 u, v 的集合.

④ 直至选了 $(n-1)$ 条边

时间复杂度: $T(n) = \underbrace{O(n \log n)}_{\text{边排序}} + \underbrace{O(n)}_{\text{并查集操作}} = O(n \log n)$.

当前变量 ($x_1:1, x_2:2, x_3:3$)
 前 (k-1) 个变量的加和和
 当前解向量

5.1. pseudo code: Function Backtrack ($k, \text{current_val}, x$):

if $k > 3$ then // 到达叶节点, 完成

output x ; return;

$\text{val_max} = \lfloor (12 - \text{current_val}) / w[k] \rfloor$; // 最大可取值

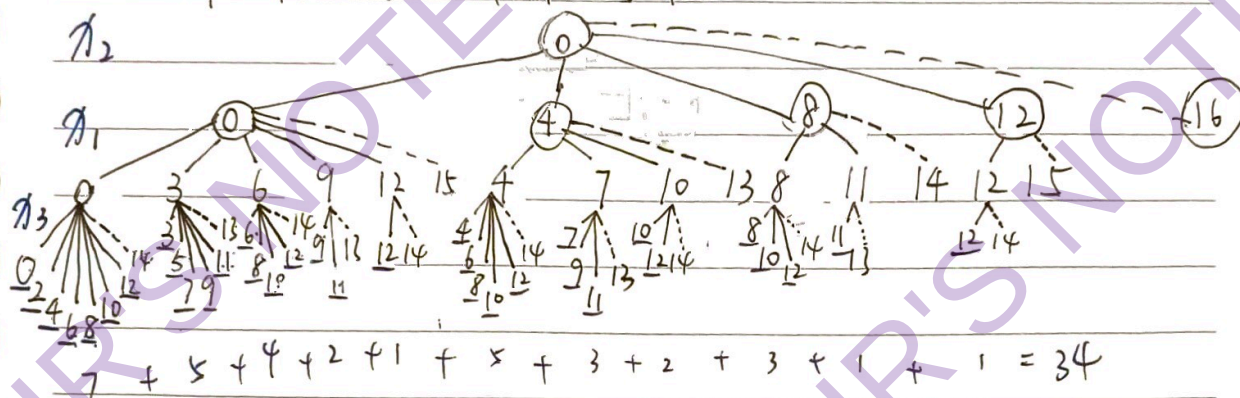
for $\text{val} = 0 \rightarrow \text{val_max}$:

$x[k] = \text{val}$;

$\text{new_val} = \text{current_val} + w[k] * \text{val}$;

Backtrack ($k+1, \text{new_val}, x$); // 递归.

求作: 调整顺序为 $4x_2 + 3x_1 + 2x_3 \leq 12$.



共 34 个叶.

5.3. ① $\langle 2, 1, 4, 3, 3, 4, 1, 2, 4, 3, 2, 1 \rangle$

② $\langle 2, 1, 4, 3, 3, 4, 2, 1, 4, 3, 1, 2 \rangle$

③ $\langle 2, 1, 4, 3, 4, 3, 1, 2, 3, 4, 2, 1 \rangle$

④ $\langle 2, 1, 4, 3, 4, 3, 2, 1, 3, 4, 1, 2 \rangle$

⑤ $\langle 2, 3, 4, 1, 3, 4, 1, 2, 4, 1, 2, 1 \rangle$

⑥ $\langle 2, 3, 4, 1, 4, 1, 2, 3, 3, 4, 1, 2 \rangle$

⑦ $\langle 2, 4, 1, 3, 3, 1, 4, 2, 4, 1, 2, 1 \rangle$

⑧ $\langle 2, 4, 1, 3, 4, 3, 2, 1, 3, 1, 4, 2 \rangle$

故, 以 2 为 $(2, 1)$ 值的解向量有 8 个. 而 $(2, 1)$ 的可能取值有 2, 3, 4, 三者性质类似.
 易知, 共有 $8 \times 3 = 24$ 个解.

No.

DATE / /

5.6. 解向量 $x = (x_1, x_2, x_3, \dots, x_n)$, $x_i \in \{0, 1\}$ 表示是否选择 i -机核.

步骤: ① $\text{sort}(S)$. ASD.

② $\text{Backtrack}(i, \text{cur_val})$:

左分支: $x_i = 1$. // 选取.

若 $\text{cur_val} + S[i] > M$, 则剪枝;

右分支: $x_i = 0$. // 不选取.

若 $\text{cur_val} + (\text{剩余值之和}) < M$, 则剪枝;

③ 不断递归, 直至 $\text{cur} == M$, 输出峰.

5.9. 分支限界法: 解空间为 k 叉树. 浮数 n .

步骤: ① $\text{sort}(t_i)$. DESD.

② 求个机器的运行时长 $(T_1, T_2, \dots, T_k)$.

③ 代所求极: $\max(T_1, T_2, \dots, T_k)$.

④ 界: $\begin{cases} LB: \max((\sum t_i)/k, \max(T_1, T_2, \dots, T_k)) \\ UB: \text{贪心算法所得解} \end{cases}$

⑤ 剪枝: 若部分解已超过当前最优解的时长, 剪枝.
若 $LB \geq \text{最优峰}$, 剪枝.

5.10. 回溯法: 记每机器的时间 F_i , 总罚款 P .

在 5.9 基础上扩展: 分配任务给 k -机器.

机器完工时间: $E_i = F_i + t_i$.

若 $E_i > B_i$, $P += f_i$.

若 $P \geq$ 当前找到的最优 P , 则剪枝.

5.11. 步骤: ①. 从左上角 (1,1) 开始遍历.

②. 若 $vis[i][j] = 1$, 该房间放置卫兵, 四周覆盖;
若 $vis[i][j] = 0$, 则予放.

③. 当处理完一行, 让 $vis[i+1][j]$ 位置未被覆盖, 则剪枝.

④. 分支限界: 维护当前卫兵数 C .

若 $C \geq C_{best}$, 则剪枝.

若 $C + C_{估计} \geq C_{best}$, 则剪枝.

其中, $C_{估计}$ 为估计到将房间需要最少卫兵数. (房间数/5).

No.

DATE / /

作业. 十二章

8.1. (1). 思想: 若 G_1 与 G_2 互补, 则边集互补. 并集为完全图.

检查 M_1 和 M_2 中 是否满足 $M_1[i][j] + M_2[i][j] = 1$,
($1 \leq i < j \leq n$.)

最坏时间复杂度: $O(n^2)$

(2). 下界: $\Omega(n^2)$.

证: 输入规模: $N = 2 \cdot n^2$.

对手论证法: 假设一算法 A , 最坏情况检查 $< \frac{n(n-1)}{2}$ 个位置就
给出"是"的结论.

那么 $\exists$ 一对 (u, v) 未检查, "是"可以修改值, 使均为 0.

而算法 A 输出 "是" 与之相悖. 故错误.

$\therefore$ 最坏时间下界为 $\Omega(n^2)$.

8.2. (1). 暴力递归法: $P(x) = a_0 + x(a_1 + x(a_2 + \dots + x(a_{n-1} + x a_n) \dots))$

递归执行 n 次, 递归第 $2n$ 次, $\Rightarrow O(n)$.

(2). (b): 反证法: 假设没有一个算法能求出系数 a_k .

那么无论取何向量, 结果一样.

然而, 只要 $a_i \neq 0$, 结果依赖于每个系数.

$\therefore$ 任何正确算法, 必须读取处理所有系数.

故下界为 $\Omega(n)$.

输入: n 个系数
 $a_0, a_1, \dots, a_n$
和变量 x .

为何 $P(x)$ 必须
取每个系数.

作业 12.5 章

1. 设 A, B, C 为判定问题. if $A \leq_p B, B \leq_p C$. then $A \leq_p C$.

ib: $\because A \leq_p B$

$\therefore \exists$ 多项式时间的函数 f , st. $\forall$ 实例 $I \in A$, 有 $f(I) \in B$.

同理: $\forall I' \in B \Rightarrow g(I') \in C$.

要ib: $\forall I \in A \Rightarrow h(I) \in C$. $\neg h(I) = g(f(I))$.

证明正确性: I 为 A 的"实例" $\Rightarrow f(I)$ 为 B 的"实例" $\Rightarrow g(f(I))$ 为 C 的"实例".
"否"实例同理.

$\Rightarrow I \in A \Leftrightarrow h(I) \in C$.

证明复杂性: 输入规模: n .

f 耗时 $O(n^{k_1})$, $\forall m = O(n^{k_1})$.

g 耗时 $O(m^{k_2}) = O((n^{k_1})^{k_2})$

$\Rightarrow h$ 耗时 $= O(n^{k_1}) + O((n^{k_1})^{k_2}) = O(n^{k_1 k_2}) = O(n^k)$

k 为常数 $\therefore h$ 为多项式时间复杂度.

$\therefore$ 得ib.

2. ib: ① 建立互斥: 对 V 的 $G=(V, E)$ 及其子集 $S \subseteq V$. 有:

S 是 G 的团 $\Leftrightarrow S$ 是 $\bar{G}$ 的独立集 $\Leftrightarrow V-S$ 是 $\bar{G}$ 的点覆盖.

② 构造归约函数: 输入: 最大团问题实例 $\langle G, k \rangle$.

变换: 构造 $\bar{G}$. (多项式时间内完成)

输出: 最大点覆盖问题实例 $\langle \bar{G}, k' \rangle$.

③ 证明正确性:

($\Rightarrow$) 若 G 有一大小为 k 的团 C , 则是 G 的独立集.

而 $V-C$ 为一个点覆盖, 大小为 $|V|-k$.

($\Leftarrow$) 若 $\bar{G}$ 有一大小为 $|V|-k=k'$ 的点覆盖 S' .

则 $V-S'$ 为 $\bar{G}$ 的独立集, 大小为 $|V|-(|V|-k)=k$.

$\therefore G$ 中, $V-S'$ 为一个大小为 k 的团.

$\therefore$ 得ib.